

Webserv Project — Complete Team Division & Architecture Guide

This document explains a complete and realistic architecture for the 42 Webserv project. It is written for beginners who are new to networking, HTTP, sockets, asynchronous I/O, CGI, and server architecture. The goal is not only to split the project between 3 teammates, but also to make sure EVERYONE understands the entire project and can explain every major concept during evaluation.

1. What Is Webserv Really?

Webserv is a simplified implementation of a real web server like NGINX or Apache.

A web server waits for browser requests, understands HTTP messages, finds requested files, and sends proper responses back to the browser.

When a browser visits a website:

Browser -> sends HTTP request -> server processes request -> server sends HTTP response.

Your project is to build this entire process manually in C++98.

2. The Core Architecture

The recommended architecture is divided into 3 clean layers:

1. Infrastructure Layer (Networking)
2. HTTP Protocol Layer
3. Application Logic Layer

This separation is extremely important because each layer has only one responsibility.

This reduces chaos, debugging complexity, and merge conflicts.

3. Data Flow of the Entire Server

The full server lifecycle should work like this:

```
Browser
↓
Person A receives raw bytes using recv()
↓
Person B parses bytes into a Request object
↓
Person C decides what to do with the request
↓
Person C creates a Response object
↓
```

Person B serializes Response into HTTP text

↓

Person A sends bytes using send()

↓

Browser receives webpage

4. Shared Structures (Very Important)

These shared structures are the contract between all teammates.
Nobody changes them alone.

Request:

- method
- uri
- query
- headers
- body
- complete

Response:

- status
- headers
- body

LocationConfig:

- path
- root
- methods
- autoindex
- upload_store
- cgi_ext
- redirect
- max_body

ServerConfig:

- host
 - port
 - error_pages
 - locations
-

5. Recommended Team Split

PERSON A — Infrastructure / Networking

PERSON B — HTTP Protocol Engine

PERSON C — Application Logic / Routing / CGI

This architecture is cleaner than mixing responsibilities.
Each teammate owns one layer completely.

6. PERSON A — Infrastructure Layer

Person A owns:

- sockets
- bind()
- listen()
- accept()
- poll()
- recv()
- send()
- client lifecycle
- timeout cleanup
- non-blocking file descriptors
- send queues
- poll event loop

IMPORTANT:

Person A does NOT understand HTTP.

To Person A, requests are only raw bytes.

Main responsibilities:

- create sockets
- monitor sockets using poll()
- receive bytes
- send bytes
- manage connections
- prevent blocking
- handle timeouts
- cleanup disconnected clients

Why this role is difficult:

- asynchronous debugging
- event-driven logic
- partial send()
- partial recv()
- race-like behavior
- freezes can break the whole server

This is usually the hardest technical role in the project.

7. PERSON B — HTTP Protocol Layer

Person B owns all HTTP knowledge.

Responsibilities:

- HTTP request parser
- request line parsing
- header parsing
- chunked transfer decoding
- partial request handling

- response serialization
- MIME type detection
- default error pages
- CGI response parsing

Person B converts:

raw bytes -> Request object
and

Response object -> raw HTTP bytes

Why this role is difficult:

HTTP formatting must be exact.

One small protocol mistake can break browser compatibility.

This role requires strong attention to details and parsing logic.

8. PERSON C — Application Logic Layer

Person C owns:

- routing
- location matching
- filesystem access
- GET handling
- POST upload handling
- DELETE handling
- redirects
- autoindex
- CGI execution
- permissions
- config parser
- testing
- config files

Why config parser was moved to Person C:

The original division gave config parsing to Person A.

This was changed because:

- config mostly affects routing
- config affects uploads
- config affects CGI
- config affects filesystem behavior
- config affects locations

All of these belong logically to the application layer.

Person A only needs:

- host
- port

Therefore config ownership fits Person C better.

9. Why Body Size Checking Was Changed

Original plan:

Person B checks max_body limits.

Problem:

The body limit depends on the matched route/location.

Route matching belongs to Person C.

Better solution:

Person B:

- parses Content-Length
- exposes body size

Person C:

- enforces max_body after route matching

This creates cleaner ownership boundaries.

10. CGI Ownership

CGI is the most dangerous integration area in the entire project.

CGI touches:

- networking
- parsing
- processes
- pipes
- HTTP
- filesystem

Correct ownership:

Person C:

- fork()
- execve()
- environment variables
- CGI setup
- timeout handling

Person A:

- monitors CGI pipe fd using poll()

Person B:

- parses CGI output as HTTP-like response

This separation avoids chaos.

11. Why One poll() Is Extremely Important

The subject requires one single poll() (or equivalent) monitoring everything.

That includes:

- server sockets
- client sockets
- CGI pipes

The poll loop is the heart of the server.

Why:

Without poll(), one slow client could freeze the entire server.

poll() allows:

- multiple simultaneous clients
 - non-blocking behavior
 - event-driven architecture
-

12. Common Beginner Mistakes

1. Starting CGI too early
 2. Coding separately without integration
 3. Changing shared structs without agreement
 4. Ignoring partial recv()
 5. Assuming send() sends everything
 6. Blocking inside event loop
 7. Forgetting cleanup of file descriptors
 8. Not testing with real browsers
 9. Not understanding HTTP format
 10. Building features before architecture
-

13. Recommended Realistic Timeline

WEEK 1

- Learn sockets and HTTP basics together
- Define architecture
- Define shared structs
- Build minimal integration skeleton

WEEK 2

Person A:

- sockets
- bind/listen
- poll loop

Person B:

- basic request parser
- response serializer

Person C:

- config parser

- route matching
- basic GET

Goal:

Browser receives a real webpage.

WEEK 3

- multiple clients
- file serving
- autoindex
- redirects
- error pages
- POST uploads
- DELETE

Goal:

Static website fully working.

WEEK 4

- CGI implementation
- chunked requests
- stress testing
- timeout handling
- browser compatibility
- edge cases

Goal:

Project stable and evaluation-ready.

FINAL DAYS

- cleanup
- README
- evaluator questions
- deep understanding sessions
- stress tests
- peer explanations

14. Integration Strategy

This is one of the most important sections.

DO NOT wait weeks before integration.

Correct strategy:

- integrate from day 1
- build minimal pipeline early
- make browser receive ANY response quickly

Example early integration:

A receives bytes

B creates fake Request

C creates fake Response

B serializes

A sends

Even if features are fake initially, the full pipeline works early.

This prevents integration disasters later.

15. Which Role Fits Different Personalities

Person A fits:

- strong Linux/system programming people
- debugging lovers
- low-level thinkers

Person B fits:

- parsing lovers
- protocol/detail-oriented people
- text processing thinkers

Person C fits:

- architecture thinkers
- big-picture learners
- people wanting full server understanding

Recommended role for the current user:

PERSON C

Reason:

- gives complete project visibility
 - easier conceptual understanding
 - strongest educational value
 - connects all server features together
-

16. Golden Rules

1. Never call `recv()/send()` without `poll()` readiness
 2. One single `poll()` for all descriptors
 3. Never block the event loop
 4. `fork()` only for CGI
 5. Every fd must be non-blocking
 6. Never leak file descriptors
 7. The server must never crash
 8. Handle partial sends and receives correctly
 9. Test constantly
 10. EVERYONE must understand EVERYTHING
-

17. Final Advice

Webserv is not mainly about websites.

The true goal is:

- understanding networking
- understanding HTTP
- understanding asynchronous servers
- understanding how browsers communicate
- understanding how real servers work internally

Do not rush.

Architecture understanding early will save enormous debugging time later.

The strongest webserv teams are not the fastest coders.

They are the teams with the cleanest architecture and communication.
